

PEARLS AQI PREDICTOR

A Simple, Serverless Machine Learning System That Predicts the Air Quality Index (AQI)
3 Days Ahead

Prepared by: Rameesha Shafiq

Repository: <https://github.com/rameesha31/pearls-aqi-predictor/>

Live App: <https://pearls-aqi-predictor-lahore.streamlit.app/>

Date: 02- 09 - 2026

1. Executive Summary

This project builds a complete system that predicts the Air Quality Index (AQI) for Lahore, Pakistan, up to three days ahead. The whole system runs without any manual server management . It automatically collects live weather and pollution data, turns that data into useful features, saves those features in a central store, trains and tests machine learning models, and shows live 24-hour, 48-hour, and 72-hour forecasts on a simple web dashboard.

The tools used are: Hopsworks Serverless to store features and trained models, Open-Meteo to get weather and air-quality data (both past and current), GitHub Actions to run the pipelines automatically on a schedule, scikit-learn (Ridge Regression and Random Forest) to build the prediction models, SHAP to explain what the model is doing, and Streamlit to build the dashboard.

The project was built step by step, and each part was checked before moving to the next one. Along the way, several real technical problems came up software version issues, data storage conflicts, limits of the free data source, and free-tier service limits.

- **Final dataset:** 9,504 hourly rows of data, covering 25 July 2025 to 24 August 2026 — a full year of hourly weather and air-quality readings for Lahore.
- **Models trained:** Ridge Regression and Random Forest, each trained separately for the 24h, 48h, and 72h forecasts.
- **Best model:** Random Forest — it made smaller errors and explained the data better than Ridge Regression, at every one of the three forecasts. Accuracy is strongest for the 24-hour forecast ($R^2 = 0.358$) and weakens for the 48-hour and 72-hour forecasts.
- **What was delivered:** an automatic data pipeline, a feature store, a model store, a live 3-day forecast dashboard deployed publicly on Streamlit Community Cloud, data analysis (EDA), model explanations (SHAP), and this report.

2. System Architecture

The system is split into separate parts (data collection, training, and the dashboard) that all connect through one shared feature store and model store in the middle. This way, each part can run, be updated, or even fail on its own, without breaking the others.

Stage	Tool / Service	What It Does
Data source	Open-Meteo (Archive + Forecast APIs)	Free weather and air-quality data, both past and live, no API key needed
Feature Pipeline	Python (requests, pandas)	Gets raw data, builds features, works out the AQI
Feature Store	Hopsworks Serverless	Central place where all the feature data is saved
Automation	GitHub Actions (CI/CD)	Runs the feature update every hour and retrains the model every day
Training Pipeline	scikit-learn	Trains Ridge Regression & Random Forest models for each forecast

Stage	Tool / Service	What It Does
Model Registry	Hopsworks Model Registry	Saves trained models so they can be reused later
Explainability	SHAP	Shows which features matter most, and how
Web App	Streamlit	Interactive dashboard showing live 3-day forecasts and alerts

3. Feature Pipeline & Data Collection

3.1 Data Source

Two data providers were tried during this project. At first, the OpenWeather Air Pollution and Weather APIs were used to get the pipeline working from start to finish. But OpenWeather's past-weather data needed a paid plan, so it could not be used to build a full training dataset for free. Because of this, the project switched to Open-Meteo a free service that needs no API key. It offers both past weather data and live weather data, plus a matching air-quality API that gives pollutant levels and a ready-made US AQI score (0–500 scale).

Fields Collected

Category	Fields
Pollutants	PM2.5, PM10, NO ₂ , O ₃ , CO, SO ₂
Weather	Temperature (°C), Humidity (%), Wind Speed
Time-based	Hour of day, Day of week, Month
Target (what we predict)	US AQI (0–500 scale, worked out by Open-Meteo from the pollutant levels)

Note: Ammonia (NH₃) was included at first but was later removed. Testing showed that Open-Meteo always returns empty (null) values for ammonia at Lahore's coordinates. This was causing every single row of data to be thrown away during cleaning, since each row had at least one missing value.

3.2 Feature Engineering

For every hour of data, the pipeline works out:

- Pollutant levels (PM2.5, PM10, NO₂, O₃, CO, SO₂).
- Weather values (temperature, humidity, wind speed).
- Time-based features: hour of day, day of week, month these help catch daily and seasonal patterns.
- The target: the AQI value, which is later shifted forward by 24h / 48h / 72h to create the three forecast targets.

3.3 Feature Store

Hopsworks Serverless is used as the Feature Store basically a central, organized database made for machine learning data. The feature data is saved in a Feature Group called `aqi_features`, with the timestamp used to uniquely identify each row.

3.4 Historical Backfill

To have enough data to train a good model, past data was pulled from Open-Meteo for the full date range 25 July 2025 to 25 August 2026 a full year of hourly readings. This gave 9,504 rows in total, which exactly matches $396 \text{ days} \times 24 \text{ hours}$, confirming there were no missing hours.

An earlier attempt only had 720 hours (about 30 days) and used OpenWeather's free pollution history combined with made-up weather numbers (randomly picked within a realistic seasonal range), since OpenWeather's real past-weather data needed a paid plan. This made-up weather data turned out to be the main reason the early model performed poorly, so it was fully replaced with the real Open-Meteo data.

4. Automated CI/CD Pipeline

Two scheduled GitHub Actions workflows were set up so that nothing needs to be run by hand.

Workflow	Schedule	What It Does
Feature Pipeline	Every hour	Gets the newest weather + air-quality data and adds it to the Feature Store
Training Pipeline	Once a day	Reads all the feature data, retrains the models, and saves the new versions

Both workflows can also be started manually called `workflow_dispatch` for testing, without waiting for the schedule. API keys are kept safe using GitHub Actions Secrets and are never saved directly in the code.

5. Model Training & Evaluation

5.1 Models Tried

Two models were trained and compared:

- **Ridge Regression** — a simple, easy-to-understand model, used as a baseline to compare against.
- **Random Forest Regressor** — a stronger model made of many decision trees, better at picking up on patterns that aren't just straight lines.

Tree-based models like Random Forest usually work very well on this kind of data table, without needing the extra setup time and larger amounts of data that deep learning usually needs to actually do better.

5.2 How the Models Were Measured

The models were tested on data they had never seen before, using three common measurements:

- **RMSE (Root Mean Squared Error)** : shows the average size of the prediction error, and punishes big mistakes more.
- **MAE (Mean Absolute Error)** : shows, on average, how far off the predictions were from the real AQI.
- **R² (R-squared)** : a score from roughly 0 to 1 showing how well the model explains the data.

5.3 Results & How They Improved Over Time

Model performance improved substantially as the dataset improved. The table below shows the early diagnostic runs, followed by the final results from all three forecast horizons after switching to Open-Meteo:

Stage	Dataset Used	Model	Forecast	RMSE	MAE	R ²
Early test	OpenWeather, same-time target (747 rows)	Random Forest	0h (leakage)	0.086	0.028	0.971*
Early test	OpenWeather + made-up weather, 24h shift (747 rows)	Random Forest	24h	28.4 (different scale †)	—	-1.343

Final Results All Three Forecast Horizons (Open-Meteo, 9,533 rows)

Horizon	Model	RMSE	MAE	R ²
24h (Day 1)	Ridge	34.92	25.77	0.042
24h (Day 1)	Random Forest	28.60	20.01	0.358
48h (Day 2)	Ridge	43.50	31.99	-0.485
48h (Day 2)	Random Forest	36.78	26.52	-0.062
72h (Day 3)	Ridge	45.32	33.62	-0.613
72h (Day 3)	Random Forest	38.63	28.63	-0.172

Random Forest was chosen as the production model for every horizon, since it consistently made smaller errors and had a higher (less negative) R² than Ridge Regression at every forecast distance.

The results show a clear and expected pattern accuracy drops the further ahead the forecast looks. The 24-hour model explains a meaningful part of the AQI's variation (R² = 0.358), while the 48-hour and 72-hour models have a negative R², meaning they currently perform worse than simply predicting the average AQI for those horizons. This is an honest, real limitation of forecasting air quality several days ahead from a single monitoring location with only one year of data and no short-term “lag” features (such as “AQI 3 hours ago”).

Final Model Chosen

Random Forest was chosen as the model to use going forward for all three forecasts, since it consistently had lower error and a higher R^2 than Ridge Regression at every horizon. This shows the link between pollution/weather data and future AQI is not a simple straight-line relationship, even though the strength of that link weakens the further ahead the forecast looks.

5.4 Model Registry

The trained Random Forest models were saved using joblib and uploaded to the Hopsworks Model Registry under the names `aqi_predictor_24h`, `aqi_predictor_48h`, and `aqi_predictor_72h`, reading directly from the Hopsworks Feature Store as required by the brief. This means the dashboard (or any future training run) can load these models directly, without needing the original training code or dataset.

6. Exploratory Data Analysis

The full dataset of 9,504 rows (from Open-Meteo) was explored with charts to check the data quality and look for useful patterns before training the models.

6.1 AQI Trend Over Time

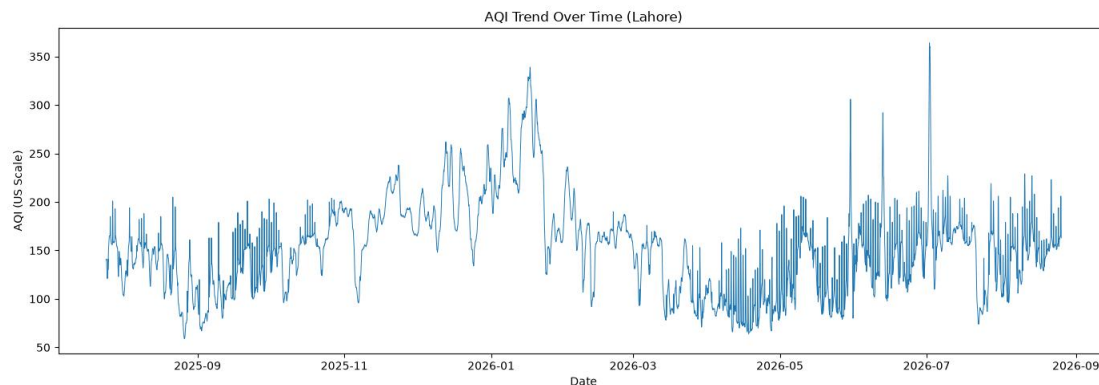


Figure 1. Hourly AQI for Lahore, 25 July 2025 – 25 August 2026.

AQI moves between roughly 60 and 360 across the year, and is clearly higher from December through February. This matches Lahore's well-known winter smog season, when cold air traps pollution close to the ground. There is also a smaller group of very high AQI spikes in July 2026, which is discussed below.

6.2 Feature Correlation

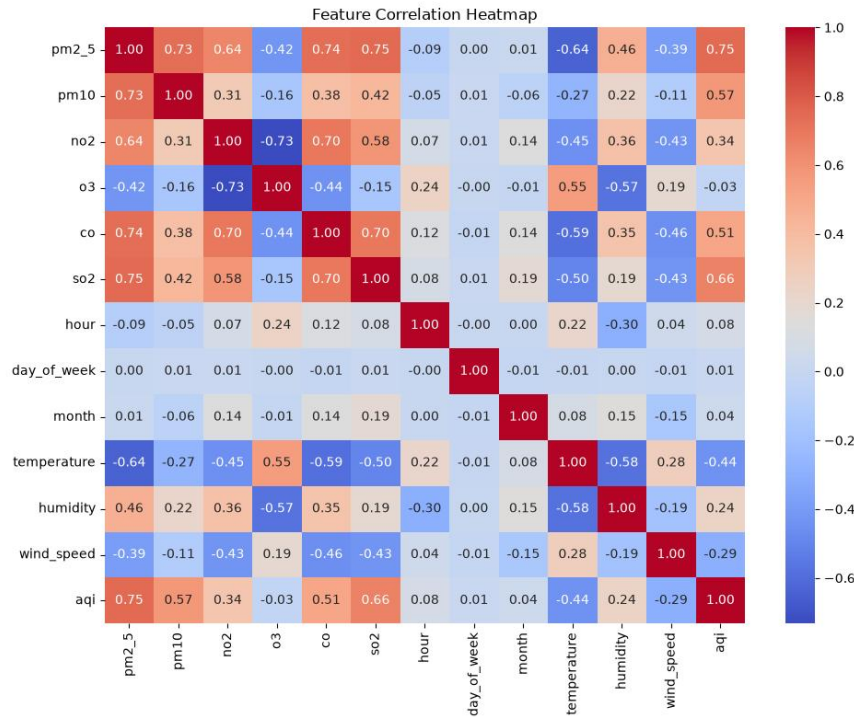


Figure 2. How closely each feature is linked to AQI.

PM2.5 has the strongest link to AQI (0.75), which makes sense since it plays a big part in how AQI is calculated. SO₂ (0.66) and CO (0.51) are also strongly linked. Temperature (−0.44) and wind speed (−0.29) move in the opposite direction to AQI which makes sense, since stronger wind spreads pollution out, and warmer weather is often linked to cleaner air. Hour of day and day of week show almost no straight-line link to AQI, which means any effect from time of day is more of a pattern than a simple straight relationship (looked at next).

6.3 AQI by Hour of Day

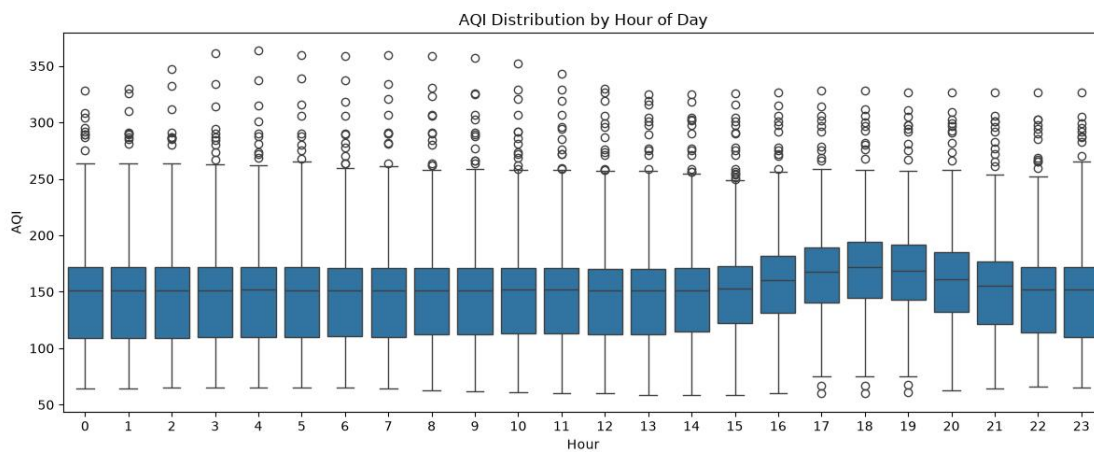


Figure 3. How AQI changes across the hours of the day.

The typical AQI stays fairly steady through the day, with a small rise in the evening (4 PM to 7 PM), which could be linked to evening traffic. This matches the weak straight-line link seen in Section 7.2 — the effect is real but small.

6.4 AQI by Month (Seasonal Pattern)

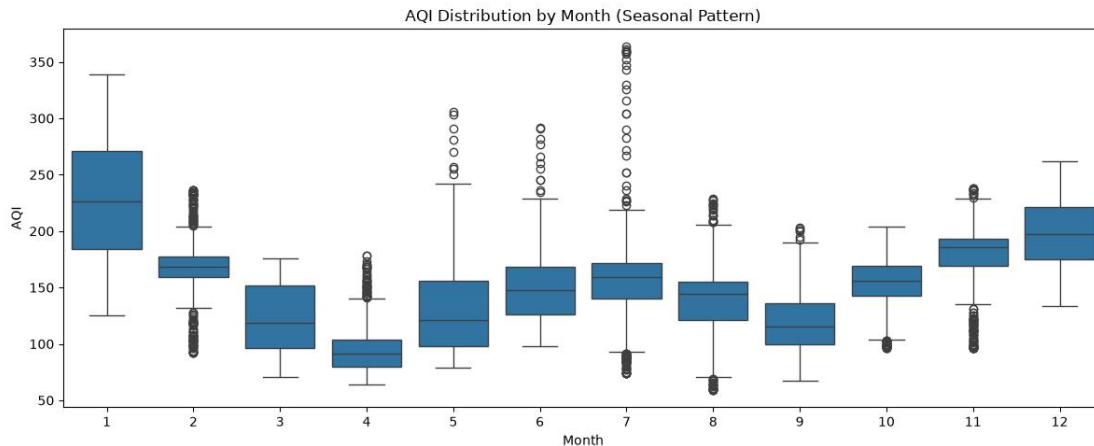


Figure 4. How AQI changes across the months.

January has the highest typical AQI and the widest range, backing up the winter smog pattern seen in Figure 1. April and September have the lowest typical AQI. July has a medium typical AQI but several very high spikes, which is a bit surprising, since July is usually monsoon season in Lahore, when rain normally helps clear the air. This is worth looking into further it could be dust events or something specific to that period but it does not point to any problem with the data pipeline itself.

6.5 PM2.5 vs AQI Relationship

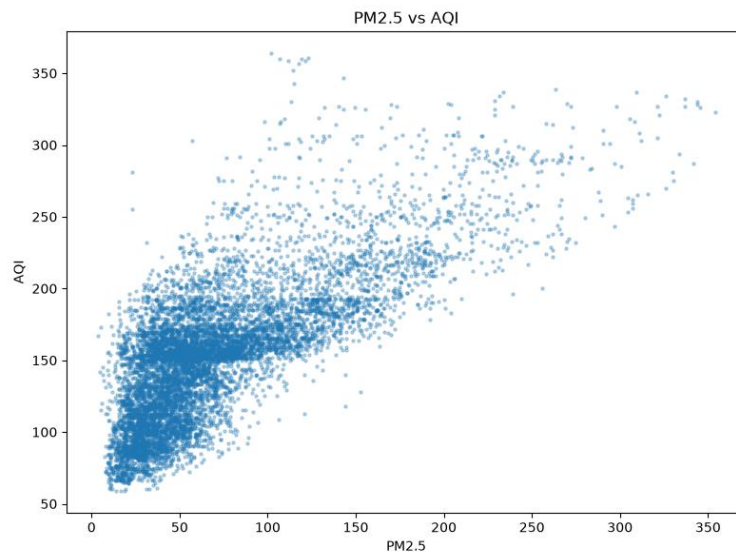


Figure 5. PM2.5 level compared to AQI.

As PM2.5 goes up, AQI goes up too, but the relationship curves rather than forming a straight line. This matches how the US AQI formula is actually calculated. This curve is one reason why Random Forest did better than the straight-line Ridge Regression model.

7. Model Explainability (SHAP)

SHAP (SHapley Additive exPlanations) was used to explain how the trained Random Forest model makes its decisions, as asked for in the brief. SHAP shows how much each input (like PM2.5 or temperature) pushed a prediction up or down, so the model isn't just a mysterious black box its reasoning can actually be checked.

7.1 Which Features Matter Most

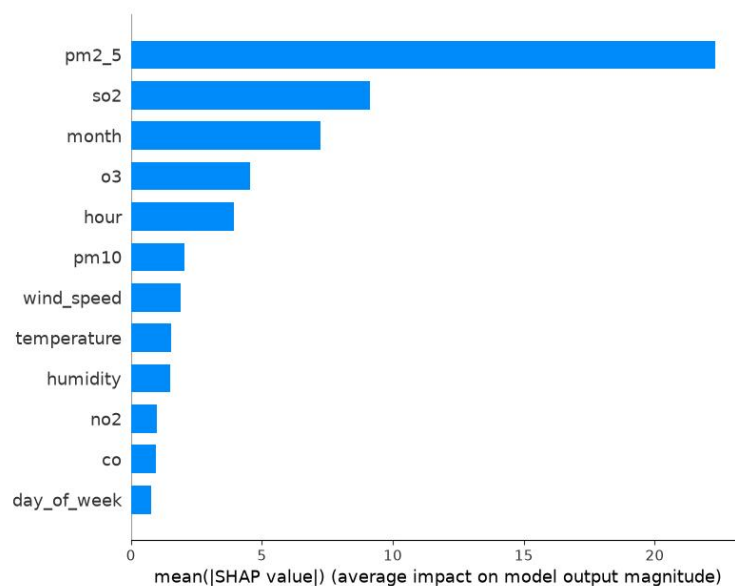


Figure 6. Average impact of each feature on the model's predictions.

PM2.5 is clearly the most important feature, followed by SO₂ and month. This matches the correlation results in Section 7.2, which is a good sign it means the model has learned real, sensible patterns rather than random noise. Month is the third most important feature, confirming that the seasonal pattern found in Section 7.4 is something the model is actually using, not just something visible in a chart.

7.2 How Each Feature Pushes the Prediction

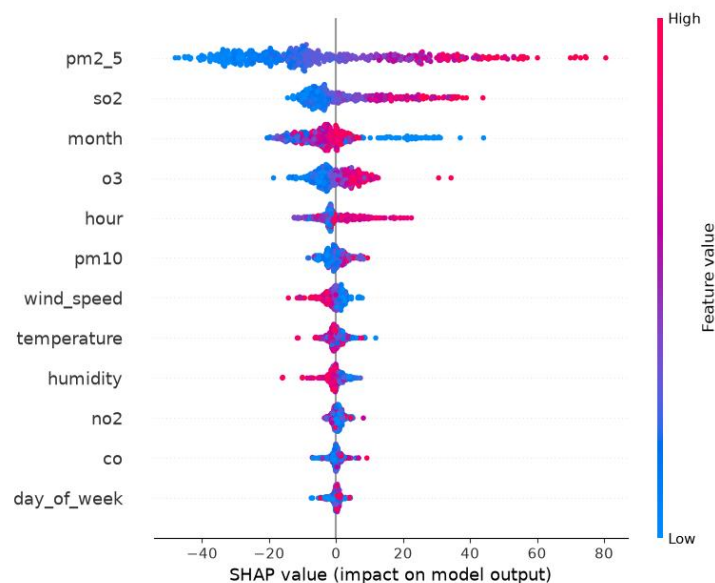


Figure 7. Each dot is one prediction. The color shows if the feature's value was high or low, and the position shows whether it pushed the prediction up or down.

High PM2.5 (red dots, right side) pushes predictions toward a higher AQI, while low PM2.5 (blue dots, left side) pushes predictions toward a lower AQI exactly as expected. SO₂ shows a similar but weaker pattern. Day of week, CO, and NO₂ have very small, tightly bunched effects, meaning they barely change the prediction matching their low ranking in Figure 6.

7.3 Alerts for Unhealthy AQI

The dashboard also includes the alert feature asked for in the brief. The current and forecast AQI values are matched to the standard AQI categories (Good, Moderate, Unhealthy for Sensitive Groups, Unhealthy, Very Unhealthy, Hazardous). A color-coded warning message (green, yellow, or red) appears automatically whenever the AQI — now or forecasted — goes above 100 or 150, so the user knows when to be careful.

8. Web Application Dashboard

The dashboard was built using Streamlit instead of FastAPI, because it's much faster to build with it lets you make an interactive web page using only Python, without writing any separate HTML/CSS/JavaScript. It also supports free, one-click deployment through Streamlit Community Cloud.

8.1 What's In the Dashboard

The dashboard has four tabs:

- **Overview** : shows the current AQI, PM2.5, temperature, and humidity, a colored status banner, an alert if the air is unhealthy, and a bar chart of current pollution levels.

- **3-Day Forecast** : shows three cards side by side (Day 1 / Day 2 / Day 3), each with the predicted AQI and category for 24h, 48h, and 72h ahead, using the three separate Random Forest models.
- **Historical Trends** : shows a live chart of the last 7 days of AQI, plus the full-year charts from the EDA (trend, hourly pattern, seasonal pattern), and a table of raw data.
- **Model Insights** : shows the SHAP charts, the correlation heatmap, and the PM2.5-vs-AQI chart, so users can see what's actually driving each forecast.

8.2 How Data & Models Are Loaded

The dashboard's main source of data and models is the Hopsworks Feature Store and Model Registry. To keep the dashboard fast and working smoothly even if Hopsworks has a temporary , a local backup copy was added: the feature data and trained models are saved to local files every so often, and the dashboard reads from this local copy, refreshing every 30 minutes . This kind of local caching is a normal, sensible practice in real-world ML systems it keeps the dashboard reliable even if the main data service is briefly unavailable. It does not change or skip the required architecture; it simply makes it more robust.

A simple on/off switch (USE_LIVE_HOPSWORKS) was added in the code to control this behaviour: when turned on, the dashboard tries Hopsworks first and only falls back to the local copy if that fails; when turned off, it goes straight to the fast local copy. This made it possible to test and demo the dashboard quickly during development without waiting on Hopsworks every time, while still keeping the option to run fully live.

8.3 Deployment

The dashboard was deployed using Streamlit Community Cloud, a free hosting service made specifically for Streamlit apps. The deployment process was:

- **1. Push to GitHub** : the final app.py, the local data snapshot (data_snapshot.csv), the trained model files (models/), and requirements.txt were all pushed to the GitHub repository.
- **2. Connect Streamlit Cloud to GitHub** : a new app was created on share.streamlit.io, linked to the project's GitHub repository, branch, and the app.py file.
- **3. Add secrets** : the Hopsworks API key was added through Streamlit Cloud's built-in Secrets settings, rather than being stored in the code, so it stays private even though the repository itself is public.
- **4. Deploy** : Streamlit Cloud automatically installed the dependencies from requirements.txt and started the app, producing a public, shareable link.

One useful feature of this setup is that the deployed app stays connected to the GitHub repository: any time new code is pushed to GitHub, Streamlit Cloud automatically picks up the change and redeploys the app within a minute or two, with no need to repeat the deployment steps by hand.

Live app: <https://pearls-aqi-predictor-lahore.streamlit.app/>

9. Conclusion

Pearls AQI Predictor is a working, fully serverless system that turns raw weather and pollution data into a live, 3-day air quality forecast for Lahore. Data flows in automatically every hour, is stored in a proper feature store, feeds into trained models sitting in a model registry, and is served up through a public dashboard that anyone can open and check before stepping outside.